Previous Years' Questions

1. 500 bytes of data are stored in memory starting at address A000H. What will be the address of the last byte? [5]
2. **Draw the block diagram of the 8086 CPU architecture with proper labels for all the components. [10]**
3. Explain any five addressing modes in the 8086. Give suitable examples using MOV instructions. [10]
4. **Write the names of any ten registers in the 8086. [5]**
5. Write the names of any five addressing modes in the 8086 with suitable examples. [5]
6. **Draw the functional block diagram of the 8086 microprocessor. [5]**
7. **What is segmented memory? [5]**
8. **Explain any five registers from the group of general purpose and index registers of the 8086 microprocessor. [5]**
9. Explain how to write ADD instruction in five different addressing modes and specify the location of the operand which is placed in the memory. [5]
10. **What is memory segmentation in the 8086 microprocessor? [5]**

Syllabus:

- Introduction to Functional Block diagram of microprocessor 8086
- MOV Instruction Formats, Addressing modes of microprocessor 8086
- Segmented memory and interleaved memory architecture in 8086

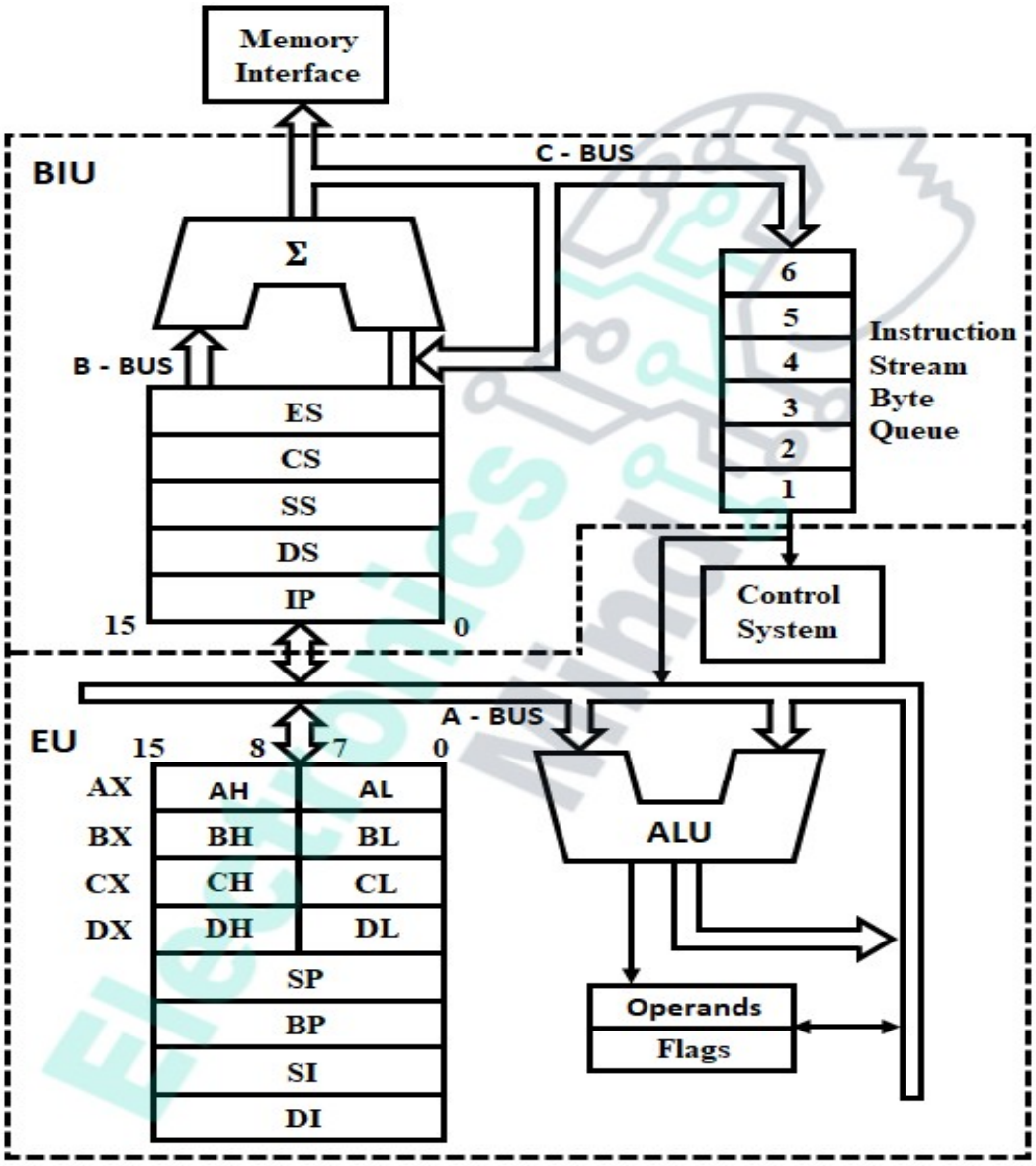
Functional block diagram of 8086

The 8086 microprocessor is an 8-bit/16-bit microprocessor designed by Intel in the late 1970s. It is the first member of the x86 family of microprocessors, which includes many popular CPUs used in personal computers.

The architecture of the 8086 microprocessor is based on the Complex Instruction Set Computer (CISC) architecture, which means that it supports a wide range of instructions, many of which can perform multiple operations in a single instruction. The 8086 microprocessor has a 20-bit address bus, which can address up to 1 MB of memory, and a 16-bit data bus, which can transfer data between the microprocessor and memory or I/O devices.

The 8086 microprocessor has a segmented memory architecture, which means that memory is divided into segments that are addressed using both a segment register and an offset. The segment register points to the start of a segment, while the offset specifies the location of a specific byte within the segment. This allows the 8086 microprocessor to access large amounts of memory, while still using a 16-bit data bus.

The 8086 CPU is divided into 2 independent functional parts to speed up the processing, namely BIU (Bus Interface Unit) & EU (Execution Unit).



8086 Internal Architecture

Bus Interface Unit (BIU):

The bus interface unit interfaces 8086 with the external world. It handles all the data transfer functions. The BIU performs read and write operations on data in the memory and on the external devices connected to the ports of the microprocessor, and it also sends out addresses. That means all external operations are performed by BIU. To perform these operations BIU contains some functional parts, which are:

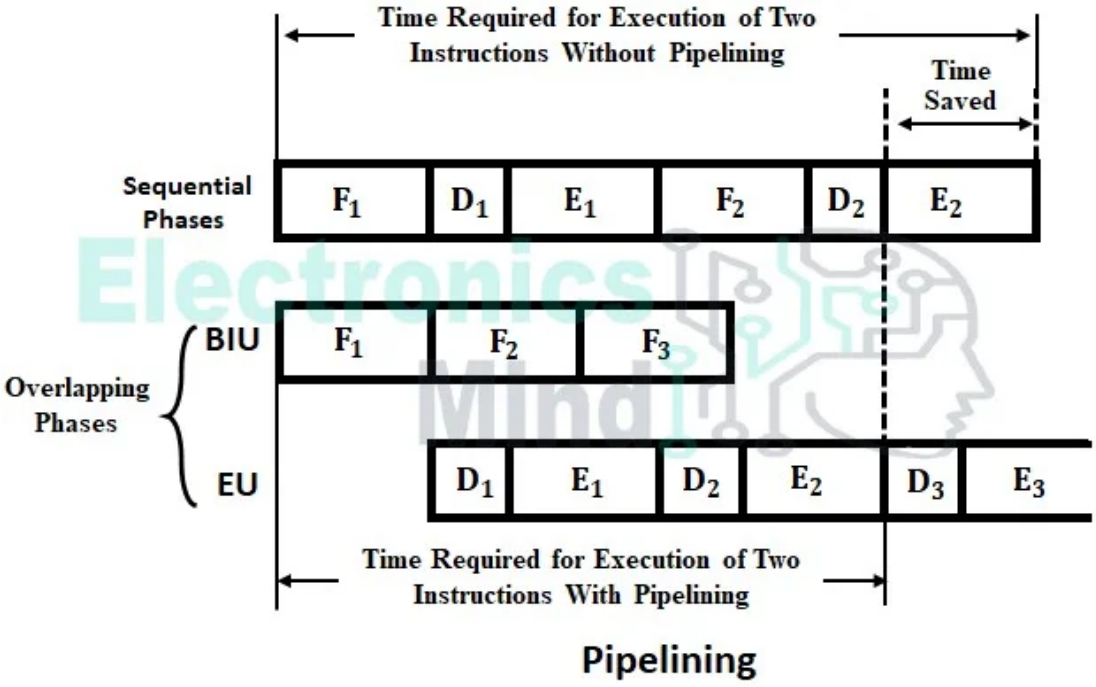
Instruction Queue :

The instruction queue holds the six bytes of instruction fetched from the code memory by the BIU that are to be executed by the EU. The instruction queue contains registers that work on FIFO (first-in-first-out) principle. Since EU and BIU are independent, whenever the execution

unit starts decoding and executing fetched instructions, the BIU fetches additional instructions to the queue from the code memory for execution.

Again when the execution unit starts executing fetched instructions in the queue, the BIU adds instructions to the queue at the same time. Since the fetching of instructions from the memory and execution of instructions in the queue are done at the same time, both the operations will overlap. This is called pipelining which speeds up the execution by reducing CPU waiting state time as illustrated in the below figure.

Since the two operations i.e. fetching an instruction from code memory by bus interface unit and execution of the instruction by execution unit are performed simultaneously, it is called parallel processing.



Pipelining

Segment Registers:
The 8086 microprocessor has 1 megabyte (220 bits = 1 Mb) of segmentized memory divided into 16 logical segments. The maximum size of each segment is 64kb so that any location within the segment can be accessed with a 16-bit logical address. At any given time, 8086 works with only four 64kb segments which are independent and separately addressable units. Each segment is associated with segment registers that are 16 bits wide. The four registers related to four segments are:

- Data Segment Register (DS) – It is used to point to the base address of the data register.
- Code Segment Register (CS) – It is used to point to the base address of the code register.
- Extra Segment Register (ES) – It holds the base address of the extra memory segment.
- Stack Segment Register (SS) – It holds the base address of the stack memory segment.

Instruction Pointer (IP):
The instruction pointer is similar to the program counter in the 8085 CPU. The IP is updated by BIU so that it contains the offset (distance in byte from base address) of the next instruction. That is IP points to the next instruction to be fetched from the code segment.

Execution Unit (EU):
The execution unit tells the bus interface unit where to fetch instruction or data from. It decodes the instructions and performs arithmetic and logical operations using the ALU. This execution unit provides effective addresses of memory locations. Then, this address is added to the base address provided by segment registers in the bus interface unit. The execution unit consists of the following parts:

Control System: It manages all the operations in the execution unit.

Instruction Decoder: It translates or decodes the instructions in the queue for execution.

Arithmetic and Logic Unit: Depending upon the given instructions the ALU performs operations like addition, subtraction, AND, OR, NOT, shifting of binary numbers, etc, on the data.

Flag Register: It is a 16-bit register with 16 flip-flops or flags. Out of 16 flags, only 9 are used. In these 9 flags, six flags are used for indicating the status of the result of an operation (status flags), and three flags are used to control certain operations in the EU (control flags).

The flag register is an important component of the 8086 microprocessor because it is used to determine the behavior of many conditional jump and branch instructions. The various flags in the flag register are set or cleared based on the result of arithmetic, logic, and other instructions executed by the processor.

Carry Flag (CF): This flag is set to 1 when there is an unsigned overflow. For example, when you add the bytes 255 and 1, the result is not in the range of 0 to 255. When there is no overflow, this flag is set to 0.

Parity Flag (PF): This flag is set to 1 when there is an even number of one bits in the result and to 0 when there is an odd number of one bits.

Auxiliary Flag (AF): Set to 1 when there is an unsigned overflow for low nibble (4 bits).

Zero Flag (ZF): Set to 1 when the result is zero. Set to 0 for a non-zero result.

Sign Flag (SF): Set to 1 when the result is negative. Set to 1 when the result is positive. This flag takes the value of the most significant bit.

Trap Flag (TF): Used for on-chip debugging.

Interrupt enable Flag (IF): When this flag is set to 1, the CPU reacts to interrupts from external devices.

Direction Flag (DF): This flag is used by some instructions to process arrays. When this flag is set to 0, the processing is done forward. When this flag is set to 1, the processing is done backward.

Overflow Flag (OF): Set to 1 when there is a signed overflow. For example, when we add the bytes 100 and 50, the sum 150 is not in the range of -128 to 127.

Its uses are:

- **Debugging**: The flag register provides a convenient way to access important information about the status of the processor after executing an instruction. This information can be used to debug programs and to optimize performance.
- **Optimization**: The flag register can be used to optimize the performance of assembly language programs by avoiding unnecessary instructions or reducing the number of conditional jumps required.
- **Error detection and handling**: The flag register can be used to detect errors and exceptions, such as overflow or divide-by-zero errors. This allows programs to handle these errors gracefully and to take appropriate corrective action.

Its advantages are:

- **Improved error handling**: The flag register can be used to detect errors and exceptions, such as overflow or divide-by-zero errors. This allows programs to handle these errors gracefully and to take appropriate corrective action.
- **Easy access to processor status information**: The flag register provides a convenient way to access important information about the status of the processor after executing an instruction. This information can be used to debug programs and to optimize performance.
- **Efficient conditional branching**: The flag register enables efficient conditional branching in assembly language programming. Programmers can use conditional jump instructions to make decisions based on the state of the flags in the flag register, allowing for more efficient and optimized code.

Its disadvantages are:

- **Limited number of flags**: The 8086 flag register has a limited number of flags, which can make it difficult to handle complex calculations or to detect certain types of errors or exceptions.
- **Limited precision**: The flags in the flag register are often limited in precision, which can lead to inaccuracies or errors in certain types of calculations or operations.
- **Overhead in execution time**: Accessing the flag register can add overhead to program execution time, which can impact performance in certain types of applications.

General Purpose Registers

The 8086 contains eight 16-bit general-purpose registers (AX, BX, CX, and DX) used to store data, variables, temporary results and can be used as counters. Each register is divided into two

8-bit registers i.e. AX as AH and AL, BX as BH and BL, CX as CH and CL, and DX as DH and DL.

AX: This is the accumulator. It is of 16 bits and is divided into two 8-bit registers AH and AL to also perform 8-bit instructions. It is generally used for arithmetical and logical instructions but in the 8086 microprocessor it is not mandatory to have an accumulator as the destination operand.

Example: ADD AX, AX (AX = AX + AX)

BX: This is the base register. It is of 16 bits and is divided into two 8-bit registers BH and BL to also perform 8-bit instructions. It is used to store the value of the offset.

Example: MOV BL, [500] (BL = 500H)

CX: This is the counter register. It is of 16 bits and is divided into two 8-bit registers CH and CL to also perform 8-bit instructions. It is used in looping and rotation.

Example: MOV CX, 0005
LOOP

DX: This is the data register. It is of 16 bits and is divided into two 8-bit registers DH and DL to also perform 8-bit instructions. It is used in the multiplication and input/output port addressing.

Example: MUL BX (DX, AX = AX * BX)

Stack Pointer (SP): It is a 16-bit register that is the stack pointer. Its size is 16 bits. It points to the topmost item of the stack. If the stack is empty then the pointer will be (FFFE)H. Its offset address is relative to the stack segment.

Base Pointer (BP): This is the base pointer. Its size is 16 bits. It is primarily used in accessing parameters passed by the stack. Its offset address is relative to the stack segment.

Source Index (SI): This is the source index register. Its size is 16 bits. It is used in the pointer addressing of data and as a source in some string-related operations. Its offset is relative to the data segment.

Destination Index (DI): This is the destination index register. Its size is 16 bits. It is used in the pointer addressing of data and as a destination in some string-related operations. Its offset is relative to the extra segment.

Index registers: SI, DI
Pointer registers: SP, BP, IP
General purpose registers: AX, BX, CX, DX, SP, BP, SI, DI
Segment registers: DS, CS, ES, SS
Flag registers: CF, PF, AF, ZF, SF, TF, IF, DF

Why is the architecture divided into BIU and EU?
In general, almost all early 8-bit CPUs used the approach of "fetch-execute-fetch" while running a program from code memory. During the time that instruction is fetched, the CPU will be in the waiting state. In order to eliminate this CPU waiting state and to increase the execution speed, the 8086 processor is composed of two sections or units that operate asynchronously.

Thus there are two independent units i.e., BIU and EU, in the 8086 microprocessor. The BIU fetches instructions from code memory whenever the memory is free. The BIU gets instructions bytes and shifts them into the internal memory called instruction queue or pipeline. The EU gets the instructions out of the queue and executes them as fast as it can.

The BIU puts instructions in one end of the pipeline and the EU takes them out at the other end of the pipeline. Thus, the BIU-EU combination can fetch and execute code simultaneously. Hence, to increase the execution speed of the processor, 8086 architecture is divided into BIU and EU.

Instruction Formats in 8086
The instruction format of 8086 has one or more number of fields associated with it. The first field is called operation code field or opcode field, which indicates the type of operation. The instruction format also contains other fields known as operand fields. There are six general formats of instructions in the 8086 instruction set. The length of an instruction may vary from one byte to six bytes.

Addressing Modes in 8086
- Addressing mode indicates a way of locating data or operands.
- Depending upon the data types used in the instruction and the memory addressing modes, any instruction may belong to one or more addressing modes, or some instruction may not belong to any of the addressing modes.
- Thus addressing modes describe the types of operands and the way they are accessed for executing an instruction.
- According to the flow of instruction execution, the instructions may be categorized as
  - Sequential control flow instructions
  - Control transfer instructions
- Sequential control flow instructions are the instructions which after execution, transfer control to the next instruction appearing immediately after it in the program. For example, the arithmetic, logical, data transfer and processor control instructions are sequential control flow instructions.
- The control transfer instructions, on the other hand , transfer control to some predefined address or the address somehow specified in the instruction, after their execution.
- For example INT, CALL, RET and JUMP instructions fall under this category.

Immediate addressing
- In this type of addressing, immediate data is a part of instruction, and appears in the form of successive byte or bytes
- Eg: MOV AX, 0005H

Direct addressing

- In the direct addressing mode, a 16-bit memory address (offset) is directly specified in the instruction as a part of it.
- Eg: MOV AX,[5000H], –Effective address= 10H*DS +5000H

Register addressing

- In the register addressing mode, the data is stored in a register and it is referred to using the particular register.
- All the registers, except IP, may be used in this mode.
- Eg: MOV AX, BX 4 Register

Indirect addressing

- Sometimes, the address of the memory location which contains data or operand is determined in an indirect way, using the offset registers.
- This mode of addressing is known as register indirect mode.
- In this addressing mode, the offset address of data is in either BX or SI or DI register.
- The default segment is either DS or ES. The data is supposed to be available at the address pointed to by the content of any of the above registers in the default data segment.
- Eg: MOV AX,[BX] –Effective address is 10H*DS+[BX]

Indexed addressing

- In this addressing mode, the offset of the operand is stored in one of the index registers.
- DS is the default segment for index registers SI and DI.
- In the case of string instructions DS and ES are default segments for SI and DI respectively.
- This mode is a special case of the above discussed register indirect addressing mode.
- Eg: MOV AX,[SI] –effective address is 10H*DS+[SI]

Register Relative addressing

- In this addressing mode, the data is available at an effective address formed by adding an 8-bit or 16-bit displacement with the content of any one of the registers BX, BP, SI and DI in the default (either DS or ES) segment.
- Eg: MOV AX,50H[BX] –Effective address is 10H*DS+50H+[BX]

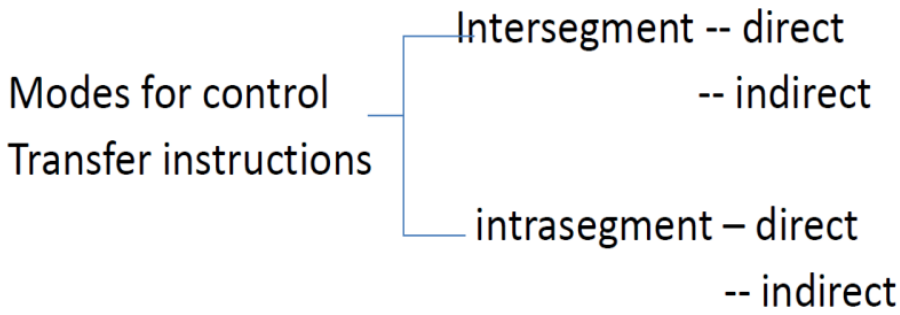
Based Indexed addressing

- The effective address of the data is formed by adding the content of a base register (any 1 of BX or BP) to the content of an index register (any 1 of SI or DI)
- The default segment register may be DS or ES.
- Eg: MOV AX,[BX][SI]
- The effective address is 10H*DS +[BX]+[SI]

Relative Based Indexed addressing

- The effective address is formed by adding an 8-bit or 16-bit displacement with the sum of contents of any one of the base registers (BX or BP) and any one of the index registers (SI or DI), in a default segment.
- Eg: MOV AX,50H[BX][SI]
- For the control transfer instructions, it depends upon whether the destination location is within the same segment or in a different one.
- It also depends upon the method of passing the destination address to the processor.
- Basically there are two addressing modes for the control transfer instructions, viz, intersegment and intrasegment addressing modes.

- If the location to which the control is to be transferred lies in a different segment other than the current one, the mode is called the intersegment mode.
- If the destination location lies in the same segment, the mode is called intrasegment mode



Memory Segmentation in 8086

The chunks that a program is divided into which are not necessarily all of the exact sizes are called segments. **Segmentation** is the process in which the main memory of the computer is logically divided into different segments and each segment has its own base address. It is basically used to enhance the speed of execution of the computer system, so that the processor is able to fetch and execute the data from the memory easily and fast.

Need for Segmentation

The Bus Interface Unit (BIU) contains four 16 bit special purpose registers (mentioned below) called segment registers, which are CS, DS, ES and SS. The number of address lines in 8086 is 20, 8086 BIU will send a 20 bit address, so as to access one of the 1MB memory locations. The four segment registers actually contain the upper 16 bits of the starting addresses of the four memory segments of 64 KB each with which the 8086 is working at that instant of time. A segment is a logical unit of memory that may be up to 64 kilobytes long. Each segment is made up of contiguous memory locations. It is an independent, separately addressable unit. Starting address will always be changing. It will not be fixed. Note that the 8086 does not work the whole 1MB memory at any given time. However, it works only with four 64KB segments within the whole 1MB memory.

Types Of Segmentation

Overlapping Segment

A segment starts at a particular address and its maximum size can go up to 64 kilobytes. But if another segment starts along with this 64kilobytes location of the first segment, then the two are said to be overlapping segments.

Non-Overlapped Segment

A segment starts at a particular address and its maximum size can go up to 64 kilobytes. But if another segment starts before this 64 kilobytes location of the first segment, then the two segments are said to be non-overlapped segments.

Rules of Segmentation

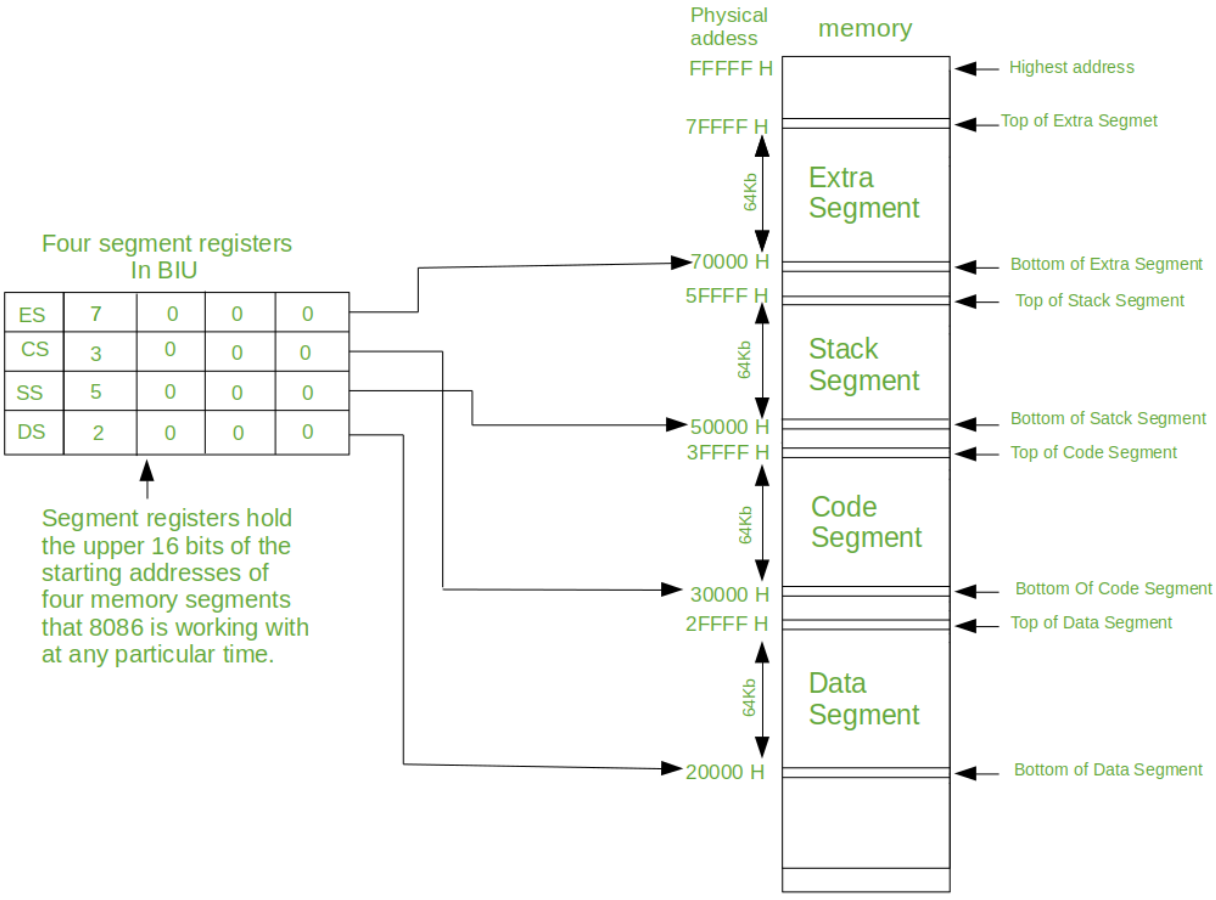
The process of segmentation follows some rules as follows:

- The starting address of a segment should be such that it can be evenly divided by 16.
- Minimum size of a segment can be 16 bytes and the maximum can be 64 kB.

Advantages of Segmentation

The main advantages of segmentation are as follows:It provides a powerful memory management mechanism.

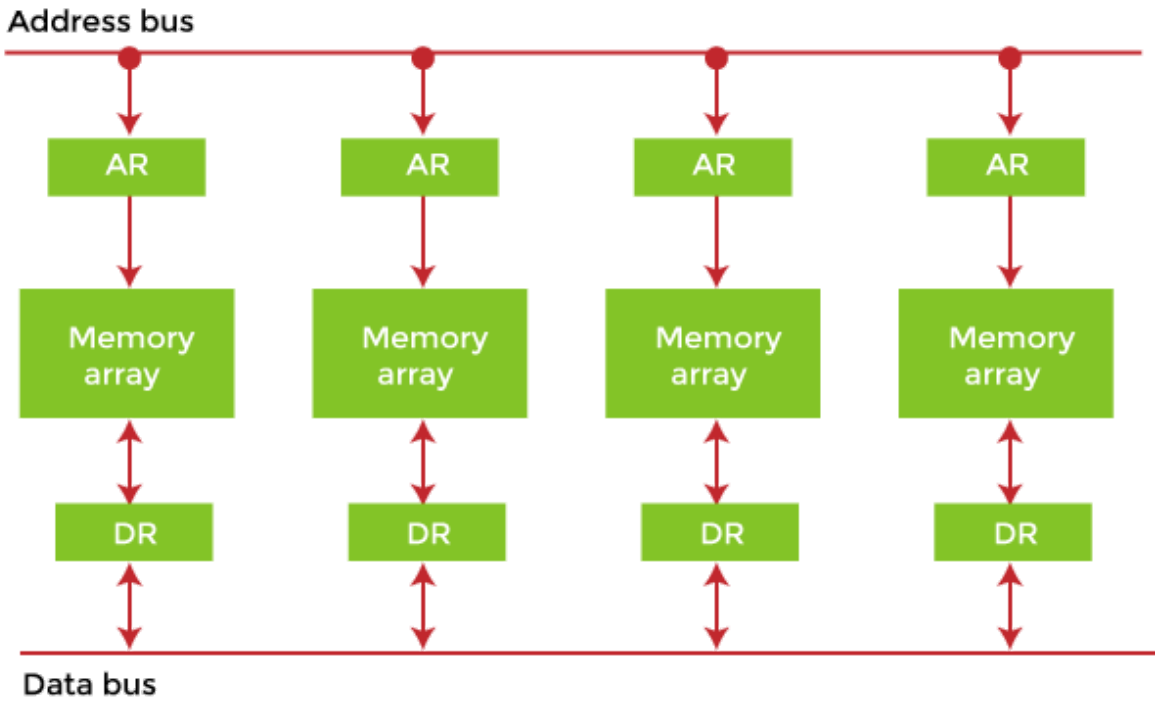
- Data related or stack related operations can be performed in different segments.
- Code related operations can be done in separate code segments.
- It allows processes to easily share data.
- It extends the address ability of the processor, i.e. segmentation allows the use of 16 bit registers to give an addressing capability of 1 megabyte. Without segmentation, it would require 20 bit registers.
- It is possible to enhance the memory size of code data or stack segments beyond 64 KB by allotting more than one segment for each area.



Interleaved Memory

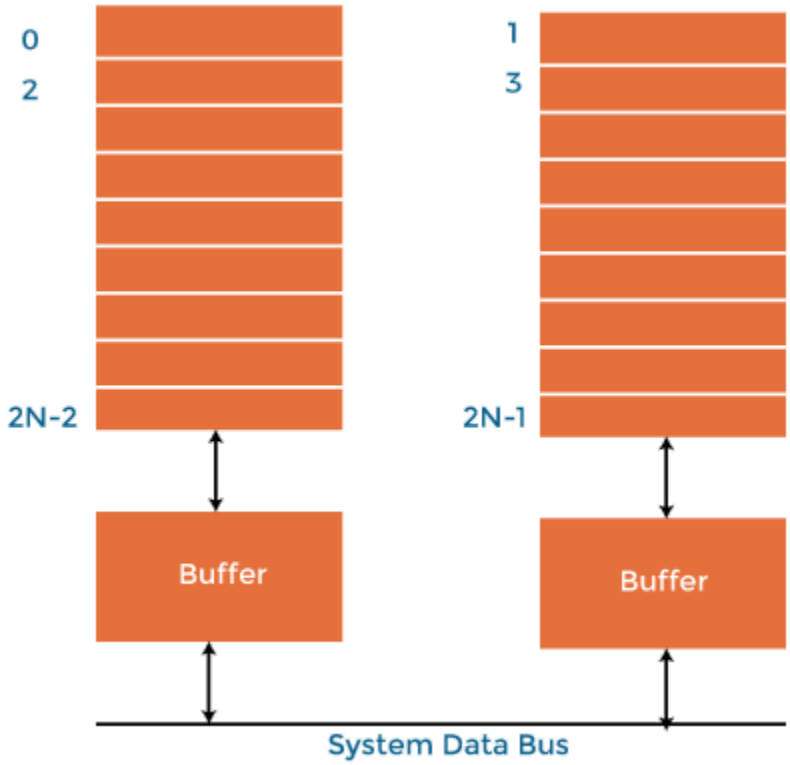
Interleaved memory is designed to compensate for the relatively slow speed of dynamic random-access memory (DRAM) or core memory by spreading memory addresses evenly across memory banks. In this way, contiguous memory reads and writes use each memory bank, resulting in higher memory throughput due to reduced waiting for memory banks to become ready for the operations.

It is different from multi-channel memory architectures, primarily as interleaved memory does not add more channels between the main memory and the memory controller. However, channel interleaving is also possible, for example, in ***Freescale*** i.MX6 processors, which allow interleaving to be done between two channels. With interleaved memory, memory addresses are allocated to each memory bank.



Example of Interleaved Memory:
It is an abstraction technique that divides memory into many modules such that successive words in the address space are placed in different modules. Suppose we have 4 memory banks, each containing 256 bytes, and then the Block Oriented scheme (no interleaving) will assign virtual addresses 0 to 255 to the first bank and 256 to 511 to the second bank. But in Interleaved memory, virtual address 0 will be with the first bank, 1 with the second memory bank, 2 with the third bank and 3 with the fourth, and then 4 with the first memory bank again. Hence, the CPU can access alternate sections immediately without waiting for memory to be cached. There are multiple memory banks that take turns for the supply of data. In the above example of 4 memory banks, data with virtual addresses 0, 1, 2 and 3 can be accessed simultaneously as they reside in separate memory banks. Hence we do not have to wait to complete a data fetch to begin the next operation.

An interleaved memory with n banks is said to be ***n-way interleaved***. There are still ***two banks of DRAM*** in an interleaved memory system, but logically, the system seems to have one bank of memory that is twice as large. In the interleaved bank representation below with 2 memory banks, the first long word of bank 0 is flowed by that of bank 1, followed by the second long word of bank 0, followed by the second long word of bank 1 and so on. The following image shows the organization of two physical banks of n long words. All even long words of the logical bank are located in physical bank 0, and all odd long words are located in physical bank 1.



Why do we use Memory Interleaving?

When the processor requests data from the main memory, a block (chunk) of data is transferred to the cache and then to the processor. So whenever a cache miss occurs, the data is to be fetched from the main memory. But the main memory is relatively slower than the cache. So to improve the access time of the main memory, interleaving is used. For example, we can access all four modules at the same time, thus achieving parallelism. The data can be acquired from the module using the higher bits. This method uses memory effectively.

Types of Interleaved Memory: In an operating system, there are two types of interleaved memory, such as:

1. High order interleaving: In high order memory interleaving, the most significant bits of the memory address decides memory banks where a particular location resides. But, in low order interleaving the least significant bits of the memory address decides the memory banks.

The least significant bits are sent as addresses to each chip. One problem is that consecutive addresses tend to be in the same chip. The maximum rate of data transfer is limited by the memory cycle time. It is also known as ***Memory Banking***.

Address bits	25-22	21-0
Use	Bank Select	Address to the chip

Module 0	Module 1	Module 2	Module 3	Module 4	Module 5	Module 6	Module 7
0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

2. Low order interleaving: The least significant bits select the memory bank (module) in low-order interleaving. In this, consecutive memory addresses are in different memory modules, allowing memory access faster than the cycle time.

Address bits	25-4	3-0
Use	Address to the chip	Bank Select

Module 0	Module 1	Module 2	Module 3	Module 4	Module 5	Module 6	Module 7
0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

Benefits of Interleaved Memory:
An instruction pipeline may require instruction and operands both at the same time from main memory, which is not possible in the traditional method of memory access. Similarly, an arithmetic pipeline requires two operands to be fetched simultaneously from the main memory. So, to overcome this problem, memory interleaving comes to resolve this.

- It allows simultaneous access to different modules of memory. The modular memory technique allows the CPU to initiate memory access with one module while others are busy with the CPU in the read or write operations. So, we can say interleave memory honors every memory request independent of the state of the other modules.
- So, for this obvious reason, interleave memory makes a system more responsive and faster than non-interleaving. Additionally, with simultaneous memory access, the CPU processing time also decreases and increases throughout. Interleave memory is useful in the system with pipelining and vector processing.
- In an interleaved memory, consecutive memory addresses are spread across different memory modules. Say, in a byte-addressable 4 way interleaved memory, if byte 0 is in first module, then byte 1 will be in the 2nd module, byte 2 will be in the 3rd module, byte 3 will be in 4th module, and again byte 4 will fall in the first module, and this continues.
- An n-way interleaved memory where main memory is divided into n-banks and the system can access n operands/instruction simultaneously from n different memory banks. This kind of memory access can reduce the memory access time by a factor close to the number of memory banks. In this memory interleaving memory location, i can be found in the bank **i mod n**.

Interleaving DRAM

Main memory is usually composed of a collection of DRAM memory chips, where many chips can be grouped together to form a memory bank. With a memory controller that supports interleaving, it is then possible to layout these memory banks so that the memory banks will be interleaved. Data in DRAM is stored in units of pages. Each DRAM bank has a row buffer that serves as a cache for accessing any page in the bank. Before a page in the DRAM bank is read, it is first loaded into the row-buffer. If the page is immediately read from the row-buffer, it has the shortest memory access latency in one memory cycle. Suppose it is a row buffer miss, which is also called a row-buffer conflict. It is slower because the new page has to be loaded into the row-buffer before it is read. Row-buffer doesn't happen as access requests on different memory pages in the same bank are serviced. A row-buffer conflict incurs a substantial delay for memory access. In contrast, memory accesses to different banks can proceed in parallel with high throughput.
In traditional layouts, memory banks can be allocated a contiguous block of memory addresses, which is very simple for the memory controller and gives an equal performance in completely random access scenarios compared to performance levels achieved through interleaving. However, memory reads are rarely random due to the locality of reference, and optimizing for close together access gives far better performance in interleaved layouts.

The way memory is addressed does not affect the access time for memory locations that are already cached, impacting only on memory locations that need to be retrieved from DRAM.